DCIT 202- MOBILE APPLICATION DEVELOPMENT

Session 1 – Introduction

Lecturer: Mr. Paul Nii Tackie Ammah, DCS
Contact Information: pammah@ug.edu.gh

UNIVERSITY OF GHANA
Department of Computer Science
School of Physical and Mathematical Sciences

Session Overview

- Goals and Objectives
  - Learn about the different types of mobile applications and the concept of cross-platform development.
  - Learn introduction to React Native, from its history to practical application.

NB: Contents of slides largely adapted from https://reactnative.dev/ and https://reactnavigation.org/

Prerequisites

- Basic understanding of programming concepts.
- Familiarity with any programming language (preferably JavaScript).

Session Outline

- Overview of Mobile App Development
- Types of mobile apps: Native, Hybrid, Web
- Introduction to cross-platform development
- Introduction to React Native

Overview of Mobile App Development

- Mobile app development is the process of creating software applications that run on mobile devices, such as smartphones and tablets.
- These applications can be pre-installed on devices, downloaded from app stores like Google Play or the Apple App Store, or accessed via mobile web browsers.
- Most commonly for the Android and iOS operating systems.

Brief History of Mobile App Development

- The evolution of mobile app development is closely tied to the advancements in mobile device technology.

1990s: Early Mobile Devices

- Basic built-in applications: calendars, calculators.
- Introduction of Personal Digital Assistants (PDAs) with third-party applications.

2008: Launch of the iOS App Store

- Revolutionized app distribution and monetization.
- Created a new industry standard for mobile apps.

Brief History of Mobile App Development

2008: Introduction of Android OS & Google Play Store

- Fostered a competitive app market with an open-source approach.
- Expanded app development to a broader audience.

Brief History of Mobile App Development

Early 2000s: Emergence of Native Apps

- Early support for native apps using platform-specific languages with Symbian OS & Palm OS.

2007-2008: Major Launches

- **iOS Introduction (2007):** Revolutionized native app development with Objective-C (later Swift).
- **Android Release (2008):** Open-source, Java-based app development, fostering broad adoption.

2010s: Advancements in Technologies

- **Swift for iOS (2014):** Easier, safer than Objective-C, enhancing app quality.
- **Kotlin for Android (2017):** Official support for cleaner, more efficient code.
- Integration of AR, machine learning, and improved security.

Brief History of Mobile App Development

Advancements and Modern Trends in Mobile App Development:

Cross-Platform Development Tools

- Emergence of frameworks like Apache Cordova, Ionic, Codename One, Xamarin, React Native, Flutter, etc
- Enabled development across multiple platforms with a single codebase.

Progressive Web Apps (PWAs)

- Mid-2010s: Web apps with app-like functionalities (offline access, push notifications).
- Blend of web and mobile app features for enhanced user experience.

Current Trends

- Integration of AI, machine learning, and augmented reality.
- Focus on security, scalability, and sustainable solutions.

Types of Mobile Apps

1. **Native Apps:**
   - Developed specifically for a particular mobile platform (iOS or Android) using platform-specific programming languages (Swift for iOS, Kotlin or Java for Android).
   - Advantages include high performance, better user experience, and full access to device features like GPS, camera, and accelerometer.
   - Disadvantages include higher development costs and the need to develop and maintain separate codebases for different platforms.
2. **Web Apps:**
   - Essentially websites that are designed to look and feel like native apps but are accessed through a mobile device's web browser.
   - They are built using web technologies such as HTML5, CSS, and JavaScript.
   - Advantages include platform independence and no need for app store approval.
   - Disadvantages include limited access to device-specific features and potentially poorer performance compared to native apps.

Types of Mobile Apps

3. Hybrid Apps:

- Blend elements of both native and web apps. They are developed using web technologies, which are then encapsulated within a native app shell.
- This allows them to be installed like a native app but also to utilize certain device features through APIs.
- Advantages include easier and faster development across multiple platforms and access to device features.
- Disadvantages may include slower performance compared to native apps and more complex debugging.

Mobile App Development Lifecycle

- **Conceptualization:** Define the purpose and goals of the app.
- **Planning:** Create a detailed app specification document and outline the app's features, functionalities, and user journey. Establish milestones and timelines.
- **Design:** Design the user interface (UI) focusing on the user experience (UX). This includes wireframing, prototyping, and final design stages, emphasizing intuitive usability and aesthetic appeal.
- **Development:** Start coding the app using the chosen development approach (native, web, or hybrid).
- **Testing:** Conduct thorough testing, including unit testing, integration testing, and user acceptance testing (UAT) to ensure the app is free from bugs and meets the initial project requirements.
- **Launch:** Deploy the app to the appropriate app stores or distribution platforms.
- **Maintenance and Updates**

Introduction to Cross-Platform Development

- Cross-platform development refers to the process of building software applications that are compatible with multiple mobile operating systems or platforms from a single codebase.
- Developers to write the code once and deploy it across various platforms, such as iOS and Android, without needing to rewrite the code specifically for each.

Advantages:

- **Cost Efficiency:** Build and maintain one application for all platforms.
- **Faster Development Time:** One codebase reduces development and update time.
- **Wider Reach:** Access more users across different platforms with one solution.

Key Tools and Technologies

1. **React Native:**
   - Uses JavaScript and React.
   - Runs on real mobile UI components.
   - Integrates with native functionality.
2. **Flutter:**
   - Utilizes Dart language.
   - Known for high-performance UI rendering.
   - Rich set of pre-designed widgets for native performance.
3. **Xamarin:**
   - Employs C# and .NET.
   - Share code across iOS, Android, and Windows.
   - Integrates seamlessly with Microsoft ecosystem.

Comparing App Development Approaches

	Performance and User Experience	Development Considerations
Native Apps	Highest performance and best user experience.	Requires separate development for each platform.
Hybrid Apps & Cross-Platform	Good performance; close to native with modern tools.	Faster development using shared code.
Web Apps	Limited by browser capabilities; consistent across platforms.	Easiest to develop, least platform integration.

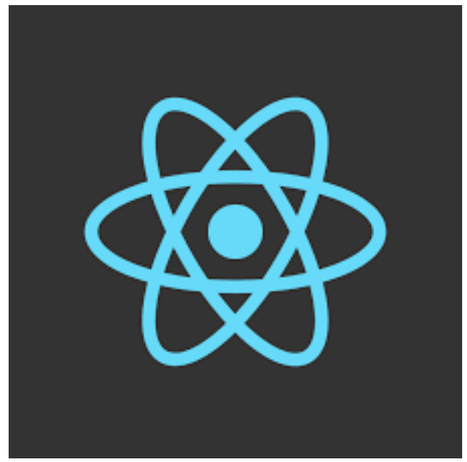
Choosing the Right Development Approach

When to Choose:

- **Native:** High-performance apps like games or advanced utilities.
- **Cross-Platform:** Cost-effective for MVPs and medium-scale enterprise apps.
- **Hybrid:** Content-driven applications needing both web presence and native store access.
- **Web Apps:** Projects with wide accessibility needs and limited budgets.

Introduction to React Native

- React Native is a framework developed by Facebook for building native apps using JavaScript.
- Allows cross-platform development for iOS and Android from a single codebase.
- React Native was officially open-sourced at Facebook's React.js conference in February 2015



History of React Native

- The core concept of using JavaScript and React to build native mobile apps emerges at a Facebook Hackathon in 2013.
- React Native is unveiled at the React.js Conference in January 2015
- Facebook makes React Native available to the public on GitHub, allowing for wider adoption and contributions on March 2015
- In September 2015, Android Support Arrives. React Native expands its reach by adding support for building Android apps alongside iOS.
- Continued Growth and Innovation in 2016 and beyond. Regular updates have introduced performance improvements, new features, and enhanced platform support.

Advantages

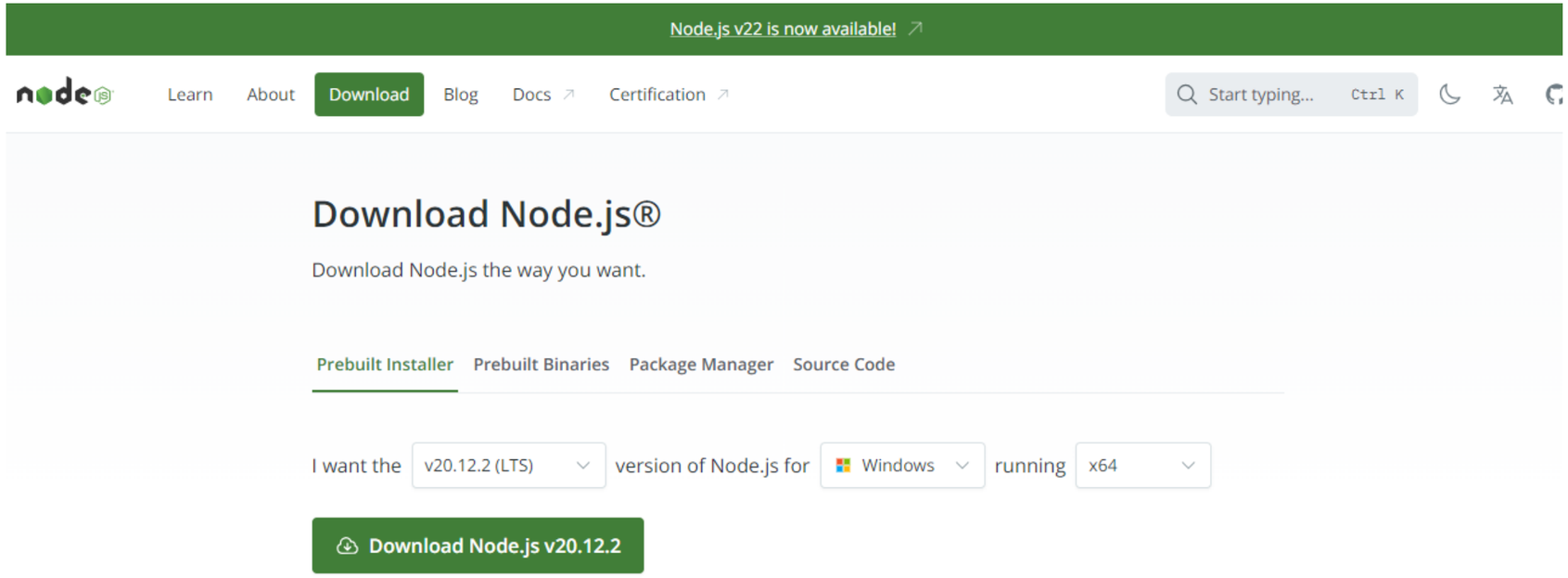
- **Cross-Platform Development:** Write once and run on both iOS and Android, reducing development time and cost.
- **Community and Ecosystem:** Large community support and a rich ecosystem of libraries and tools.
- **Performance:** While not as high as native apps, React Native offers close to native performance through the use of native components.
- **Live and Hot Reloading:** Streamlines the development process by allowing developers to see changes almost instantly without recompiling the app.

Real-World Applications

- Social Media (Instagram and Facebook)
- E-commerce and On-Demand Services (Walmart, Delivery.com)
- Entertainment and Fitness Apps (SoundCloud Pulse, MyFitnessPal)
- Other Notable Apps (Bloomberg, Skype, Airbnb, etc)

Setup & Installation

- React Native requires you to have Node.js (Node)
  - JavaScript runtime built on Chrome's V8 JavaScript engine
- Expo CLI or React Native CLI



Expo CLI

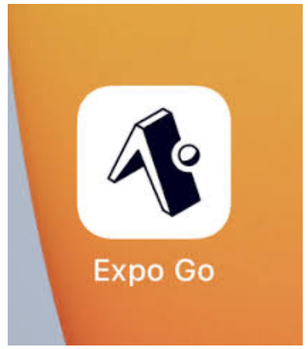
- Expo CLI is a command line app
- Used for variety of tasks including
  - Creating new projects
  - Developing your app: running the project server, viewing logs, opening your app in a simulator
  - Publishing your app JavaScript and other assets and managing releasing them over the air
  - Building binaries (apk and ipa files) to be uploaded to the App Store and Play Store
  - Managing Apple Credentials and Google Keystores

Expo CLI Installation

- **Expo CLI** is part of the expo package, and you can use it by leveraging **npx** — a Node.js package runner.
- After installing Node.js, you can use **npx** to create a new app.
- To create a new application use
  - **npx create-expo-app --template <app-name>**

Running React Native App with Expo

- Install the **Expo client** app on your iOS or Android phone from their respective app stores.
- Connect to same wireless network as computer
- With Expo you can run the app on browser, Expo client, android simulator, iOS simulator (if android and/or iOS emulators are installed)
- Using the expo app scan the QR code that is displayed after running the following command
  - *yarn start*



Modifying React Native App

- Open **App.js** in your text editor of choice and edit some lines.
- Include a View and Text component to display "Hello, World!".

```
import React from 'react';
import { View, Text } from 'react-native';


const App = () => (
  <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
    <Text>Hello, World!</Text>
  </View>
);


export default App;
```

Modifying React Native App

- The application should reload automatically once you save your changes.

Assignment

- Assignment
  - Find assignments in Sakai